

Práctica de Numpy

Arrays, operaciones vectorizadas, estadísticas y arrays 2D · Python

- Arrays
- Indexado y slicing
- Operaciones vectorizadas
- Estadísticas
- Arrays 2D

SISTEMA	Python 3 · VS Code
ARCHIVO	practica_numpy.py
TIEMPO ESTIMADO	60 – 80 minutos, individual
REQUISITO PREVIO	Tema 3, Clase 5 (Numpy)
OBJETIVO	Practicar arrays y operaciones vectorizadas en situaciones propias, con un desafío final y un reto abierto que integran todo lo visto, sin código de solución
ENTREGA	Archivo practica_numpy.py corriendo sin errores, de principio a fin, incluyendo el Desafío Final y el Reto Abierto

Cómo usar esta práctica

Esta práctica refuerza los conceptos de la Clase 5: arrays de Numpy y su ventaja frente a las listas normales. Vas a practicar con situaciones distintas a las de la clase, para confirmar que entendiste el concepto, no solo el ejemplo.

CÓMO TRABAJAR CADA PASO

Recordatorio breve: 2 o 3 líneas que refrescan el concepto.

Instrucción: la situación específica que tienes que resolver, escribiéndola tú mismo antes de mirar la solución.

Pista: un empujón técnico hacia el enfoque correcto, para intentarlo una vez más antes de ver la solución completa.

Solución y salida esperada: para autoevaluarte después de intentarlo, no para copiar de entrada.

EL DESAFÍO FINAL Y EL RETO ABIERTO SON DISTINTOS

Al cierre, combinas todo lo aprendido en un caso nuevo, sin código de solución, primero con 2 casos de prueba (Desafío Final), y después sin que se te diga qué función usar (Reto Abierto).

NOTA SOBRE EL ARCHIVO

Todo el código de esta práctica va en un solo archivo: `practica_numpy.py`. Puedes ir agregando cada paso debajo del anterior, o comentando los pasos anteriores con `#` mientras trabajas en uno nuevo.

Arrays: creación e indexado

Recordatorio: `np.array()` convierte una lista en un array. El acceso por posición funciona igual que en las listas.

INSTRUCCIÓN

Crea un array `calificaciones` con 6 notas de examen (invéntalas). Imprime la primera nota, la última, y las notas de la posición 2 a la 5 (sin incluir la 5). Imprime también cuántas notas hay en total.

No copies el ejemplo de temperaturas de la clase. Aquí son calificaciones de examen.

SOLUCIÓN (CON DATOS DE EJEMPLO, LOS TUYOS SE VERÁN DISTINTOS)

```
import numpy as np

calificaciones = np.array([88, 92, 76, 95, 81, 68])

print(calificaciones[0])
print(calificaciones[-1])
print(calificaciones[2:5])
print(len(calificaciones))
```

SALIDA ESPERADA

```
88
68
[76 95 81]
6
```

Operaciones vectorizadas

Recordatorio: multiplicar un array por un número aplica esa operación a cada elemento a la vez, sin bucle. Con una lista normal, multiplicarla solo repite la lista completa.

INSTRUCCIÓN

Crea un array `comisiones` con 4 montos de comisión de ventas (invéntalos). Aplica un bono del 10% a todas a la vez (multiplica por 1.10), sin usar ningún bucle, y redondea el resultado a 2 decimales.

Pista

Recuerda que una operación sobre un array de Numpy se aplica a todos sus elementos a la vez, sin necesidad de un `for`. Inténtalo antes de mirar la solución.

SOLUCIÓN

```
comisiones = np.array([120.50, 340.00, 89.75, 210.25])

comisiones_con_bono = comisiones * 1.10
print(comisiones_con_bono.round(2))
```

SALIDA ESPERADA

```
[132.55 374.  98.72 231.28]
```

POR QUÉ ESTO NO FUNCIONARÍA IGUAL CON UNA LISTA NORMAL

Si `comisiones` fuera una lista normal de Python (no un array), `comisiones * 1.10` daría un error, porque Python no sabe multiplicar una lista completa por un número decimal. Esa es exactamente la ventaja de un array: las operaciones matemáticas se aplican directamente sobre todos los valores.

Funciones estadísticas

Recordatorio: `np.mean()`, `np.max()`, `np.min()` y `np.std()` calculan estadísticas sobre todo el array de una sola vez.

INSTRUCCIÓN

Crea un array `consumo_gb` con el consumo de datos móviles (en GB) de 7 días distintos (invéntalos). Imprime el promedio, el máximo, el mínimo, y la desviación estándar, redondeados a 2 decimales.

Pista

Cada una de estas 4 estadísticas tiene su propia función de Numpy, ya las viste en el recordatorio de arriba. No necesitas ningún bucle ni cálculo manual.

SOLUCIÓN

```
consumo_gb = np.array([2.3, 5.1, 1.8, 8.4, 3.6, 4.2, 6.9])

print("Promedio:", round(np.mean(consumo_gb), 2))
print("Máximo:", np.max(consumo_gb))
print("Mínimo:", np.min(consumo_gb))
print("Desviación estándar:", round(np.std(consumo_gb), 2))
```

SALIDA ESPERADA

```
Promedio: 4.61
Máximo: 8.4
Mínimo: 1.8
Desviación estándar: 2.21
```

Interpreta el resultado

El consumo promedio fue 4.61 GB, con una desviación estándar de 2.21. ¿Qué te dice ese número sobre qué tan parecidos o distintos fueron tus 7 días entre sí? Si la desviación estándar fuera mucho más baja (por ejemplo, 0.3), ¿qué diría eso sobre tu consumo diario?

Arrays de 2 dimensiones

Recordatorio: `axis=1` calcula por fila, `axis=0` calcula por columna.

INSTRUCCIÓN

Con esta matriz, donde cada fila es un estudiante y cada columna es una semana de horas de estudio, calcula el promedio de horas de cada estudiante, y el promedio de cada semana entre todos los estudiantes.

Pista

Antes de decidir qué `axis` usar para cada cálculo, pregúntate cuántos resultados esperas: ¿uno por estudiante, o uno por semana? Eso te dice cuál dirección tienes que colapsar.

DATOS DE PARTIDA

```
horas_estudio = np.array([
    [5, 8, 6, 7],
    [10, 9, 11, 8],
    [3, 4, 2, 5],
])
```

SOLUCIÓN

```
print(horas_estudio.shape)
print("Promedio por estudiante:", np.mean(horas_estudio, axis=1))
print("Promedio por semana:", np.mean(horas_estudio, axis=0))
```

SALIDA ESPERADA

```
(3, 4)
Promedio por estudiante: [6.5 9.5 3.5]
Promedio por semana: [6. 7. 6.33333333 6.66666667]
```

CÓMO CONFIRMAR QUE ELEGISTE EL AXIS CORRECTO

Hay 3 estudiantes, así que el promedio por estudiante debe dar exactamente 3 números. Hay 4 semanas, así que el promedio por semana debe dar exactamente 4 números. Si tus resultados tienen la cantidad de números al revés, invertiste el `axis`.

Ventas semanales de 3 sucursales

Sin código de ejemplo esta vez, y sin solución. Combina indexado, operaciones vectorizadas y arrays 2D, exactamente como en los 4 pasos anteriores, pero en un solo caso más completo.

INSTRUCCIÓN

Con esta matriz de ventas (3 sucursales, 5 días de la semana):

```
ventas = np.array([
    [1200, 1350, 980, 1420, 1100],
    [850, 920, 1050, 890, 960],
    [1600, 1750, 1580, 1690, 1720],
])
```

Calcula: (1) el total de ventas de cada sucursal en la semana, (2) una comisión del 5% sobre el total de cada sucursal, y (3) el total de ventas combinadas de las 3 sucursales, por cada día de la semana.

Piensa qué `axis` te da un resultado por sucursal, y cuál te da un resultado por día, antes de escribir el código.

CÓMO AUTOEVALUARTE, SIN VER LA SOLUCIÓN

Compara tu salida contra este caso de prueba. Si coincide, tu lógica está correcta.

CASO DE PRUEBA

Totales por sucursal: [6050 4670 8340]

Comisiones (5%): [302.5 233.5 417.]

Totales por día: [3650 4020 3610 4000 3780]

SI TU RESULTADO NO COINCIDE

Imprime `ventas.shape` primero, y confirma que entiendes cuál dimensión (0 o 1) representa las sucursales y cuál los días, antes de revisar el resto de tu código.

Un paso más, sin que te diga qué función usar

Esta vez no te digo qué función usar, solo el resultado que necesitas encontrar. No es más difícil que el Desafío Final, es el mismo tipo de trabajo, con menos indicaciones.

INSTRUCCIÓN

Usando la misma matriz ventas del Desafío Final: determina cuál de las 3 sucursales tiene las ventas **más consistentes** a lo largo de la semana (la que menos varía día a día, no la que más vende). Imprime el resultado de las 3 sucursales para poder comparar.

Piensa en cuál de las 4 funciones estadísticas que ya usaste en el Paso 3 mide "variación", no "total" ni "promedio".

CÓMO AUTOEVALUARTE

Compara tu salida contra este caso de prueba.

CASO DE PRUEBA

Sucursal 1: desviación estándar = 160.5

Sucursal 2: desviación estándar = 68.29

Sucursal 3: desviación estándar = 66.75

LO QUE ESTE RETO CONFIRMA

Si llegaste a este resultado sin que nadie te dijera qué función usar, acabas de demostrar algo más importante que saber la función: supiste reconocer qué pregunta estadística te estaban haciendo, y elegir la herramienta correcta tú mismo.

Autoevaluación

Marca cada punto solo si lo escribiste tú mismo, sin mirar la solución antes de intentarlo.

- ☐ Creé mi propio array del Paso 1 sin copiar el ejemplo de la clase
- ☐ Entiendo por qué una operación vectorizada no funciona igual con una lista normal
- ☐ Respondí la pregunta de interpretación del Paso 3 antes de seguir
- ☐ Sé elegir correctamente entre `axis=0` y `axis=1` según cuántos resultados espero
- ☐ Mi Desafío Final coincide con el caso de prueba, sin haber visto código de solución
- ☐ Mi Reto Abierto coincide con el caso de prueba, sin que me dijeran qué función usar
- ☐ Mi archivo `practica_numpy.py` corre de principio a fin sin ningún error

SI TE TRABASTE EN ALGÚN PASO

Vuelve a repasar el concepto correspondiente de la Clase 5 antes de la próxima sesión. Numpy se sigue usando en cada clase que sigue, incluyendo cuando retomen Pandas más adelante.